

Nexus Portal and Nexus Registry Ubuntu Testing Guide

Complete step-by-step validation guide for testing the local Dockerized Nexus Portal, PostgreSQL-backed portal state, Nexus Repository Manager, Docker registry workflows, artifact publishing, Security Center, Registry Access Control, audit logs, reports, and operational status.

Generated: 2026-07-16 19:13:02

Test host IP: 172.20.102.182

1. Environment and Service URLs

Use these URLs and registry ports for the current localhost deployment.

```
Portal:      http://localhost:8095
Nexus UI:    http://localhost:8081
Grafana:     http://localhost:3000
Prometheus: http://localhost:9090
```

```
Nexus Docker hosted registry: localhost:5000
Nexus Docker proxy registry:  localhost:5001
Nexus Docker group registry:  localhost:5002
```

Ubuntu host IP for network testing: 172.20.102.182

Before running commands, load the local project environment values.

```
cd /home/parsa/project/airgapped-enterprise-artifact-platform
source .env
```

2. Verify Platform Health

```
docker compose ps
curl -fsS http://localhost:8095/health
curl -fsS http://localhost:8081/service/rest/v1/status
curl -I http://localhost:3000/login
```

Expected result: portal, PostgreSQL, and Nexus are healthy; portal health returns database=postgresql; Nexus and Grafana respond successfully.

Open the portal in a browser and sign in using PORTAL_ADMIN_USER and PORTAL_ADMIN_PASSWORD from .env.

3. Configure Docker on Ubuntu for Nexus Registry

Because this local setup uses HTTP registry ports, Docker must trust the Nexus registry as an insecure registry unless you place Nexus behind HTTPS.

```
sudo nano /etc/docker/daemon.json
```

Use this JSON content, merging with any existing Docker daemon settings if needed.

```
{
  "insecure-registries": [
    "localhost:5000",
    "localhost:5001",
    "localhost:5002",
    "172.20.102.182:5000",
    "172.20.102.182:5001",
    "172.20.102.182:5002"
  ]
}
```

```
sudo systemctl restart docker
docker info | grep -A20 "Insecure Registries"
```

Expected result: localhost and 172.20.102.182 registry addresses appear under Docker insecure registries.

4. Login to Nexus Docker Registry

```
echo "$NEXUS_ADMIN_PASSWORD" | docker login localhost:5000 -u admin --password-stdin
echo "$NEXUS_ADMIN_PASSWORD" | docker login 172.20.102.182:5000 -u admin --password-stdin
```

Expected result: Login Succeeded for both localhost and IP-based registry addresses.

5. Manual Docker Push and Pull Validation

```
docker pull alpine:3.20
docker tag alpine:3.20 localhost:5000/test/alpine:3.20
docker push localhost:5000/test/alpine:3.20
docker rmi localhost:5000/test/alpine:3.20 alpine:3.20
docker pull localhost:5000/test/alpine:3.20
docker images | grep "localhost:5000/test/alpine"
```

Expected result: image pushes to Nexus hosted registry, can be removed locally, then pulled back from Nexus.

6. Portal Docker Hub Fetch Workflow

1. Open Artifact Publishing in the portal.
2. Use Container Image by Name.
3. Set Source image to alpine:3.20.
4. Set Target image path to test/portal-alpine:3.20.
5. Set Repository to docker-hosted.
6. Click Publish Image.
7. Open Publishing History and click View Logs for the job.

Expected result: job status becomes **SUCCESS** and logs show pull, tag, login, and push operations.

```
docker pull localhost:5000/test/portal-alpine:3.20
docker images | grep portal-alpine
```

7. Upload Locally Exported Docker Image

```
docker pull nginx:1.27-alpine
docker tag nginx:1.27-alpine local/nginx-upload-test:1.0.0
docker save -o /tmp/nginx-upload-test.tar local/nginx-upload-test:1.0.0
ls -lh /tmp/nginx-upload-test.tar
```

1. Open Artifact Publishing.
2. Use Container Image Archive.
3. Set Target image to test/nginx-upload-test:1.0.0.
4. Set Repository to docker-hosted.
5. Select /tmp/nginx-upload-test.tar.
6. Click Upload Archive.
7. Review Publishing History and View Logs.

Expected result: archive is loaded, tagged, pushed to Nexus, and the job finishes successfully.

```
docker pull localhost:5000/test/nginx-upload-test:1.0.0
```

8. Python Package Fetch and Publish

1. Open Artifact Publishing.
2. Use Python Package Fetch.
3. Set Package name to requests.
4. Leave Version blank or set a known version such as 2.32.4.
5. Set Runtime to Python 3.
6. Set Repository to pypi-hosted.

7. Keep Include dependencies enabled.
8. Click Fetch & Publish.
9. Open Publishing History and click View Logs.

Expected result: pip downloads package distributions, Nexus upload succeeds, status is SUCCESS, and checksum metadata is recorded.

Validate through Nexus UI: open <http://localhost:8081>, browse pypi-hosted, and confirm package files are present.

```
python3 -m venv /tmp/nexus-pip-test
source /tmp/nexus-pip-test/bin/activate
pip install --trusted-host localhost --index-url
http://localhost:8081/repository/pypi-hosted/simple requests
python -c "import requests; print(requests.__version__)"
deactivate
```

9. Debian Package Fetch and Publish

1. Open Artifact Publishing.
2. Use Debian Package Fetch.
3. Set Package name to curl.
4. Leave Version blank.
5. Select Ubuntu 24.04 Noble or Debian 12 Bookworm.
6. Set Repository to apt-internal-hosted.
7. Keep Include dependencies enabled.
8. Click Fetch & Publish.
9. Open Publishing History and click View Logs.

Expected result: apt metadata is fetched, .deb files are downloaded, Nexus upload succeeds, and job status is SUCCESS.

Validate in Nexus UI by browsing apt-internal-hosted and confirming .deb artifacts are present.

10. Publishing Logs and History

1. Open Artifact Publishing.
2. Confirm Publish Jobs, Running, Success, and Failed counters update.
3. Click View Logs on every publish job type you tested.
4. Confirm readable progress, success output, or error messages are displayed.

Expected result: no dead buttons; logs appear in the Publishing Logs panel; failures are readable.

11. Deployment History and Reports

1. Open Deployment Wizard.
2. Select available Docker, Python, or Debian artifacts.
3. Proceed through preflight/security steps as applicable.
4. For local testing, keep remote deployment disabled if you do not want to deploy to another host.
5. Create the job.
6. Open History & Reports.
7. Test View Logs, HTML Report, PDF Report, and Refresh.

Expected result: job appears in history, logs display, and HTML/PDF reports open successfully.

12. Security Center: Access Groups

1. Open Security Center.
2. Create Group Name: Test Pullers.

3. Description: Temporary test group for approved pull clients.
4. Select docker_pull permission.
5. Click Save Group.
6. Click Edit, add pypi_read, and save again.

Expected result: group appears in Existing Groups, permissions update, and success feedback appears.

13. Security Center: Trusted Account

1. Create Username / Service ID: ci-test-puller.
2. Account Type: Service account.
3. Display Name: CI Test Puller.
4. Email: ci-test-puller@example.local.
5. Enter a test password, or leave blank to generate one.
6. Keep Enabled checked.
7. Assign Test Pullers or Registry Readers.
8. Click Save Account and save the generated password if one is shown.
9. Click Sync Nexus Access.

Expected result: Nexus role/user sync succeeds and anonymous access is disabled.

```
docker logout localhost:5000 || true
echo 'THE_TEST_PASSWORD' | docker login localhost:5000 -u ci-test-puller --password-stdin
docker pull localhost:5000/test/portal-alpine:3.20
```

Expected result: pull-only account can login and pull.

```
docker tag alpine:3.20 localhost:5000/test/should-not-push:1.0.0
docker push localhost:5000/test/should-not-push:1.0.0
```

Expected result: Expected result for pull-only user: push is denied with unauthorized or 403.

14. Publisher Permission Test

1. Create group Test Publishers.
2. Select docker_pull, docker_push, raw_read, and raw_write.
3. Create account ci-test-publisher with a known password.
4. Assign Test Publishers.
5. Click Sync Nexus Access.

```
docker logout localhost:5000 || true
echo 'PUBLISHER_PASSWORD' | docker login localhost:5000 -u ci-test-publisher --password-stdin
docker tag alpine:3.20 localhost:5000/test/publisher-push:1.0.0
docker push localhost:5000/test/publisher-push:1.0.0
docker rmi localhost:5000/test/publisher-push:1.0.0
docker pull localhost:5000/test/publisher-push:1.0.0
```

Expected result: publisher account can push and pull.

15. Anonymous and Unauthorized Blocking

After Sync Nexus Access, anonymous Nexus access should be disabled.

```
docker logout localhost:5000 || true
curl -i http://localhost:5000/v2/
echo 'wrong-password' | docker login localhost:5000 -u ci-test-puller --password-stdin
docker pull localhost:5000/test/portal-alpine:3.20
```

Expected result: unauthenticated registry access returns 401; wrong password fails; unauthenticated pull is denied.

16. Trusted IP Rules and Firewall Enforcement

1. In Security Center, add Rule Name: Admin Workstation.
2. Set IP / CIDR to 172.20.102.182/32.
3. Description: Local Ubuntu test workstation.
4. Keep Enabled checked.
5. Also add Rule Name: Localhost with IP / CIDR 127.0.0.1/32 if testing locally.
6. Click Preview Firewall.

Expected result: preview contains allow rules for trusted CIDRs followed by a DROP rule for protected ports 8081, 5000, 5001, and 5002.

Important: Apply IP Firewall only after confirming your current admin IP is trusted. A wrong CIDR can block your access to registry ports.

1. Click Apply IP Firewall.
2. Confirm Enforcement Status changes to APPLIED.

```
curl -i http://172.20.102.182:5000/v2/
```

Expected result: Expected result from approved host: HTTP 401 Unauthorized. This is good: network access is allowed, authentication is required.

```
echo "$NEXUS_ADMIN_PASSWORD" | docker login 172.20.102.182:5000 -u admin --password-stdin
docker pull 172.20.102.182:5000/test/portal-alpine:3.20
```

17. Untrusted Client Blocking

Use a second machine or VM that is not in the trusted IP list.

```
curl -m 5 -i http://172.20.102.182:5000/v2/
docker login 172.20.102.182:5000 -u admin
```

Expected result: Expected result from untrusted client: timeout, connection failure, or blocked traffic. If it receives HTTP 401, the firewall did not block it and only authentication blocked it.

```
sudo iptables -S PORTAL_NEXUS_ACL
sudo iptables -S | grep PORTAL_NEXUS
```

18. Security Center Edit/Delete Workflow

1. For Groups, Trusted Accounts, and Trusted IP Rules: create a record, edit it, save it, clear the form, delete it, then refresh the page.
2. Confirm edit forms populate correctly.
3. Confirm delete asks for confirmation and removes the record.
4. Confirm disabled accounts remain visible but cannot authenticate after sync.

19. Audit Log

1. Open Audit Log.
2. Click Refresh.
3. Look for LOGIN_SUCCESS, CREATE_SERVER, DELETE_SERVER, CREATE_IP_RULE, DELETE_IP_RULE, CREATE_ACCESS_GROUP, CREATE_ACCESS_PRINCIPAL, SYNC_NEXUS_ACCESS, and CREATE_JOB entries.

20. Storage Guard

1. Open Storage Guard.
2. Click Refresh.
3. Confirm workspace, Nexus, PostgreSQL portal database, bundle, report, and cleanup retention metrics are visible.
4. Run Cleanup Policy only if you are comfortable deleting old generated bundles/reports.

21. Troubleshooting Commands

```
docker logs airgap-portal --tail=200
docker logs airgap-nexus --tail=200
docker compose ps postgres
docker exec airgap-postgres pg_isready -U "$POSTGRES_USER" -d "$POSTGRES_DB"
docker exec airgap-postgres psql -U "$POSTGRES_USER" -d "$POSTGRES_DB" -c '\dt'
ss -lntp | grep -E '5000|5001|5002|8081|8095'
docker info | grep -A20 "Insecure Registries"
sudo iptables -S | grep PORTAL_NEXUS
sudo iptables -S PORTAL_NEXUS_ACL
docker logout localhost:5000 || true
docker logout 172.20.102.182:5000 || true
```

22. Common Failure Patterns

Docker says server gave HTTP response to HTTPS client: add the registry to Docker insecure-registries and restart Docker.

Portal publishing job fails pulling from Docker Hub or apt/pip sources: verify internet access from the portal container and check portal logs.

Docker login succeeds for untrusted client after firewall apply: verify the client IP is not accidentally covered by a trusted CIDR and inspect PORTAL_NEXUS_ACL rules.

Pull-only user can push: verify the account is assigned only to a pull-only group, then click Sync Nexus Access again.

Nexus API errors during sync: confirm Nexus is healthy and NEXUS_ADMIN_PASSWORD in .env is correct.

23. Expected Final Result

1. Portal works at <http://localhost:8095>.
2. Nexus works at <http://localhost:8081>.
3. Docker push and pull work for authorized users.
4. Portal publishing jobs succeed for Docker image fetch, Docker archive upload, Python package fetch, and Debian package fetch.
5. Publishing logs, history, reports, audit log, and Storage Guard are visible.
6. Security Center create/edit/delete workflows work.
7. Nexus anonymous access is disabled after sync.
8. Unauthorized users cannot authenticate.
9. Pull-only users cannot push.
10. Publisher users can push and pull.
11. Trusted IPs can reach registry ports, while untrusted IPs are blocked at network level.